

GW1-1 AIの3つのアプローチ — 表を完成させる（10分）

「どこに注目するか（特徴量）」と「どう判定するか（判定ルール）」を，人が作るのか，データから学習するのか，右のチップを表に貼り，空欄を埋める。

アプローチ	主な時期	特徴量 (どこに注目するか)	判定ルール (どう判定するか)	代表的な手法	チップ（ドラッグして貼る）	
ルールベース						人が設計
						人が設計
						人が設計
						人が設計
従来の機械学習 (統計的機械学習)						データから学習
						データから学習
						データから学習
						データから学習
深層学習 (深層ニューラルネットワーク)						データから学習
						データから学習
						データから学習
						データから学習

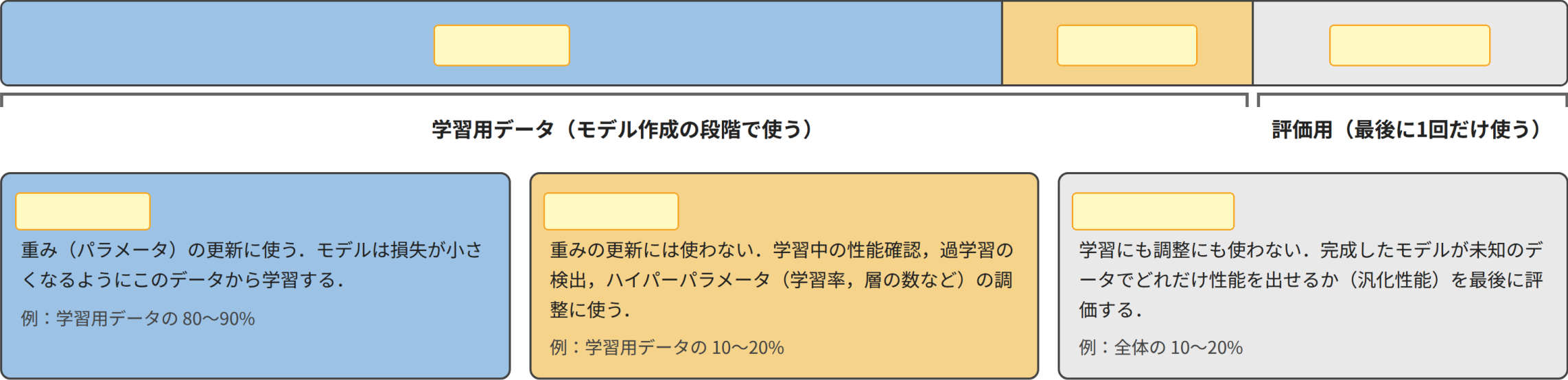
※ 深層学習の行は，特徴量と判定ルールの 2 欄を 1 つとして考えてよい

- 問い
- (1) 「人が設計」から「データから学習」へ移ったのは，表のどの欄が先か。
 - (2) 深層学習でも人が決めているものは何か（3つ）。
 - (3) 3つのアプローチは置き換わったのか，使い分けられているのか，身近な例を1つ挙げる。

GW1-2 データの分割

空欄に「訓練データ」「検証データ」「テストデータ」を入れる。説明文が手がり。

データの分割：訓練データ・検証データ・テストデータ

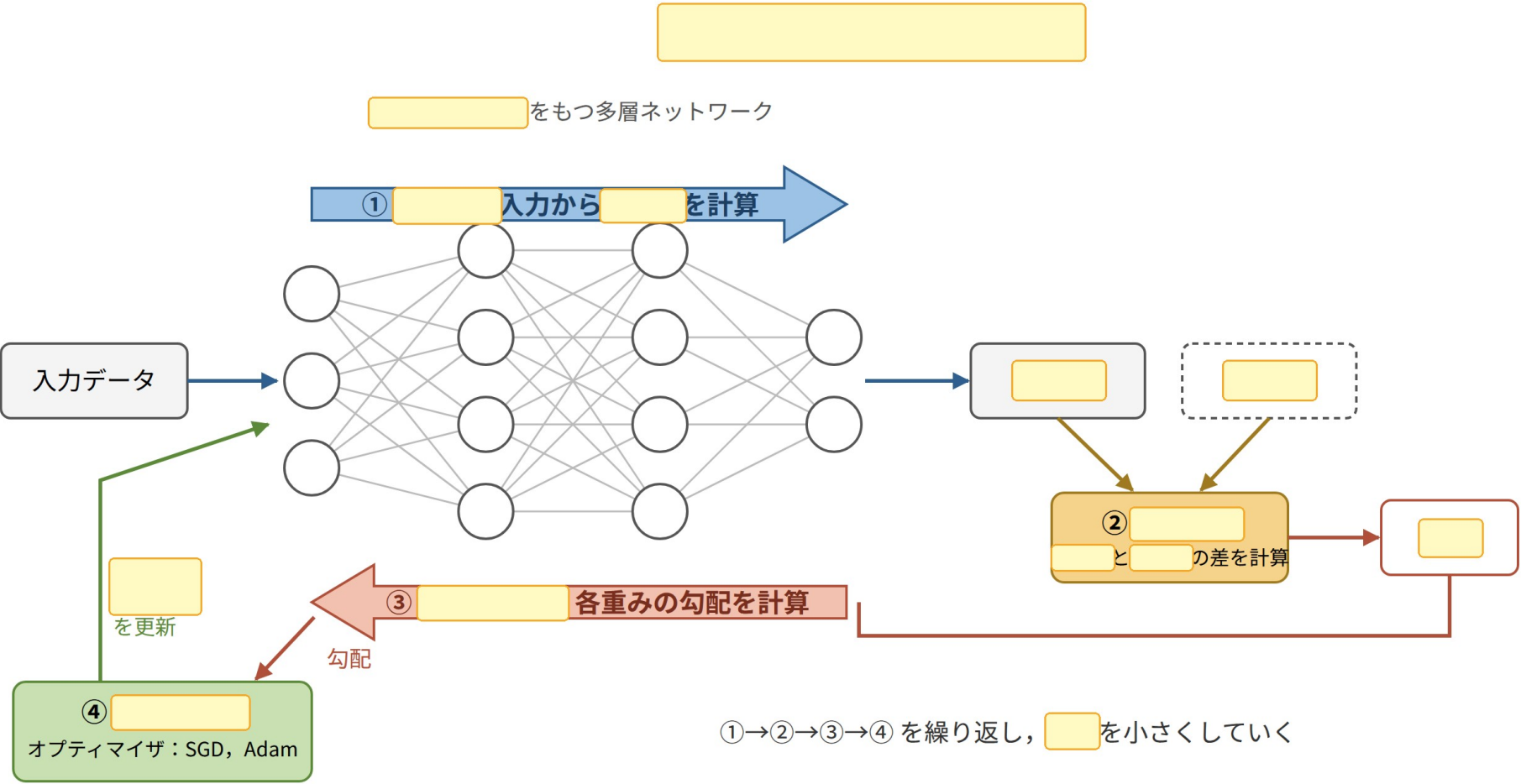


※ 比率は目安であり，データ量によって変わる（例：訓練：検証：テスト＝7：1.5：1.5，6：2：2 など）。データが少ない場合は交差検証（クロスバリデーション）を用いることもある。

※ テストデータの結果を見てモデルを調整すると，テストデータに合わせ込んだことになり，性能を正しく評価できなくなる。分類では，各クラスの割合がそろるように分割する（層化分割）。

GW1-3 順伝播と誤差逆伝播

①～④の名前, 予測値・正解値・損失, 重み・バイアス, 図のタイトルを入れる. 同じ言葉は同じ欄に入る.



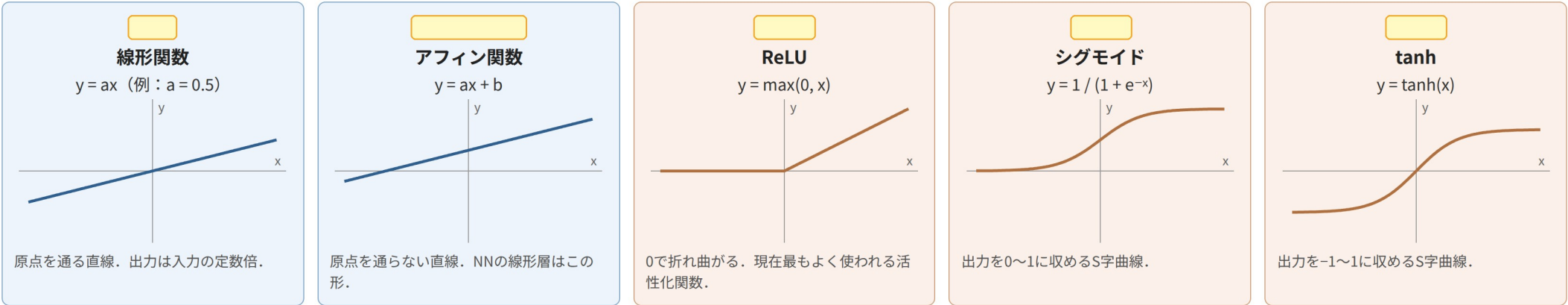
GW1-4 線形関数と非線形関数

各グラフに「線形」か「非線形」かを入れる。③の空欄は層の中の関数の名前。

① 関数：入力が決まると出力がただ1つに決まる対応関係

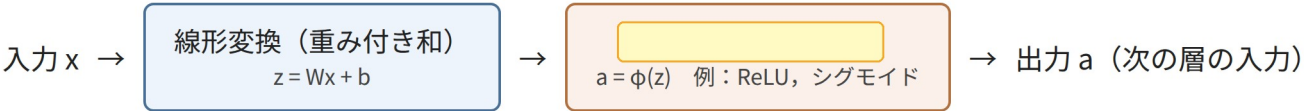


② 線形関数と非線形関数



※ 厳密には原点を通る $y = ax$ が線形関数、 $y = ax + b$ はアフィン関数である。ニューラルネットワークでは $Wx + b$ の層を慣習的に「線形層」と呼ぶ。入力が多次元の場合、グラフは直線ではなく平面（超平面）になる。

③ ニューラルネットワークの1層の計算



活性化関数に非線形関数を用いる理由：線形変換だけを何層重ねても、全体は1回の線形変換と同じになる（ $W_2(W_1x) = (W_2W_1)x$ ）。
層の間に非線形関数を挟むことで、曲線的・複雑な入出力の関係を表現できるようになる。

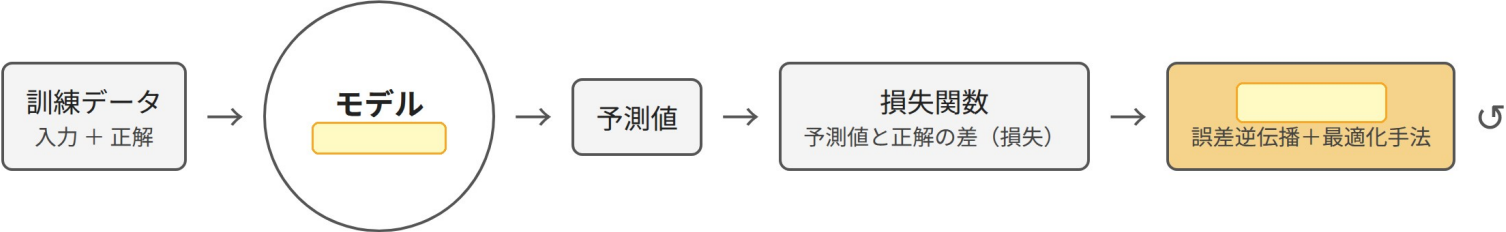
GW1-5 学習と推論

2つの段階の名前、重みの状態（更新中／固定），図のタイトルを入れる。



モデルを作る段階

入力データと正解の組を使い，予測が正解に近づくように重み（パラメータ）を少しずつ更新する。

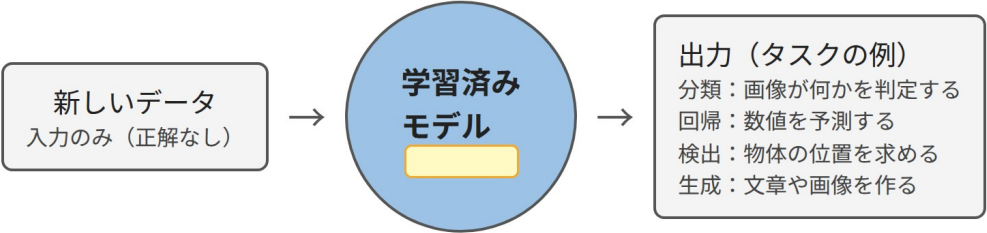


⌚ 損失が十分小さくなるまで繰り返す。検証データで性能を確認しながら，学習を止める時期やハイパーパラメータを決める。



学習済みモデルを使う段階

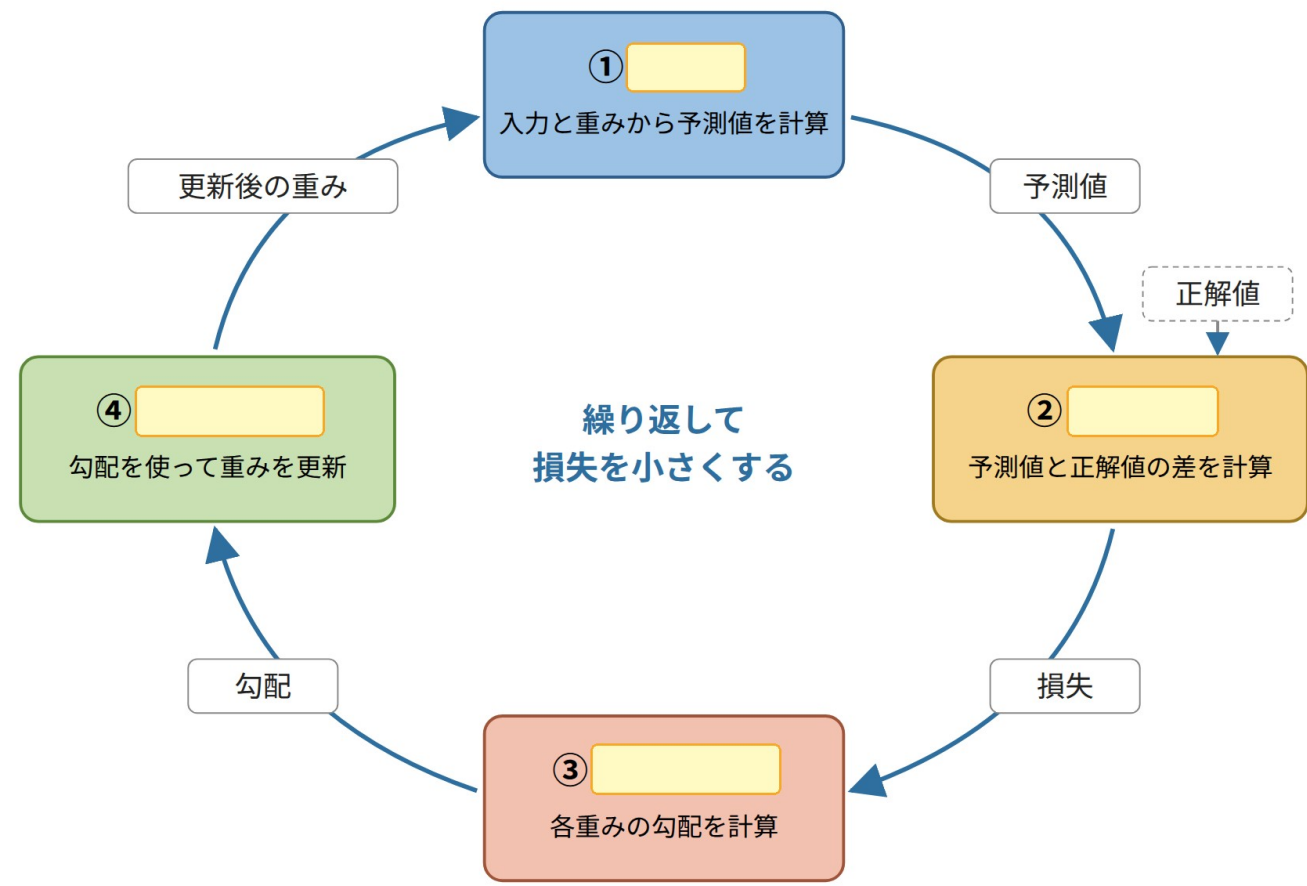
重みを固定したまま，正解の分からない新しいデータに対して出力を求める。重みは更新しない。



GW1-6 学習の1ステップ

①～④の名前を入れる。右の表の PyTorch のコードが手がかかり（GW3 のカードにも同じコードがある）。

学習の1ステップ（1回の重み更新）



段階	PyTorch のコード（例）
勾配のリセット	<code>optimizer.zero_grad()</code>
①	<code>outputs = model(inputs)</code>
②	<code>loss = criterion(outputs, labels)</code>
③	<code>loss.backward()</code>
④	<code>optimizer.step()</code>

GW2-1 プログラミングの流れ ― カードを3つのフェーズに並べる (10分)

下の12枚のカードを、正しい順に上の枠へドラッグする。番号は書いていない。並べたあと、各フェーズの見出しも埋める。

データ準備フェーズ

学習設定フェーズ

学習, 評価フェーズ

損失関数の定義

ネットワークの定義

ライブラリのインポート

データの前処理

モデル学習&検証

学習結果の可視化

デバイスの設定

最適化関数の定義

データセットの準備

データローダの作成

モデル性能評価

入力データの形状確認

- 問い (1) 「学習」が始まるのは何番目のカードからか。それより前は何をしているか。
(2) データが FashionMNIST からオリジナル画像に変わったとき、差し替えるカードはどれか。

GW2-2 入力データはどんなテンソルになるか（5分）

前処理とテンソルの形状から、右の候補を選んで左の「入力データ」の列に書く（ドラッグしてもよい）。どんなデータも「前処理で数値化 → テンソル → ディープラーニングモデル → 出力」の同じ流れに乗る。

入力データ	前処理（数値化）	テンソルの形状（例）	入力データの候補
	欠損処理・標準化	(N, 特徴量数)	画像
	トークン化→ID列→埋め込み	(N, トークン数, 埋め込み次元)	時系列
	サンプリング／スペクトログラム化	(N, サンプル数) / (N, 1, 周波数, 時間)	テキスト
	画素値の正規化・リサイズ	(N, チャンネル, 高さ, 幅)	グラフ
	フレーム抽出	(N, フレーム数, チャンネル, 高さ, 幅)	数値データ（表形式）
	座標の正規化・点の抽出	(N, 点数, 3) / 頂点(V,3)+面(F,3)	動画
	ノード特徴・接続関係の数値化	ノード特徴(ノード数, D)+隣接行列	音声
	窓切り出し・標準化	(N, 時間ステップ数, 変数数)	3D：点群／メッシュ

- 問い (1) Nは何を表すか。今日の演習で Nを変えると何が変わるか（時間？正解率？）。
- (2) FashionMNIST（白黒 28×28）とオリジナル画像（カラー 64×64）で「モデルに入る直前の形状」はどう違うか。